Lab 03 Manual

Haleemah Anwary
Seneca College

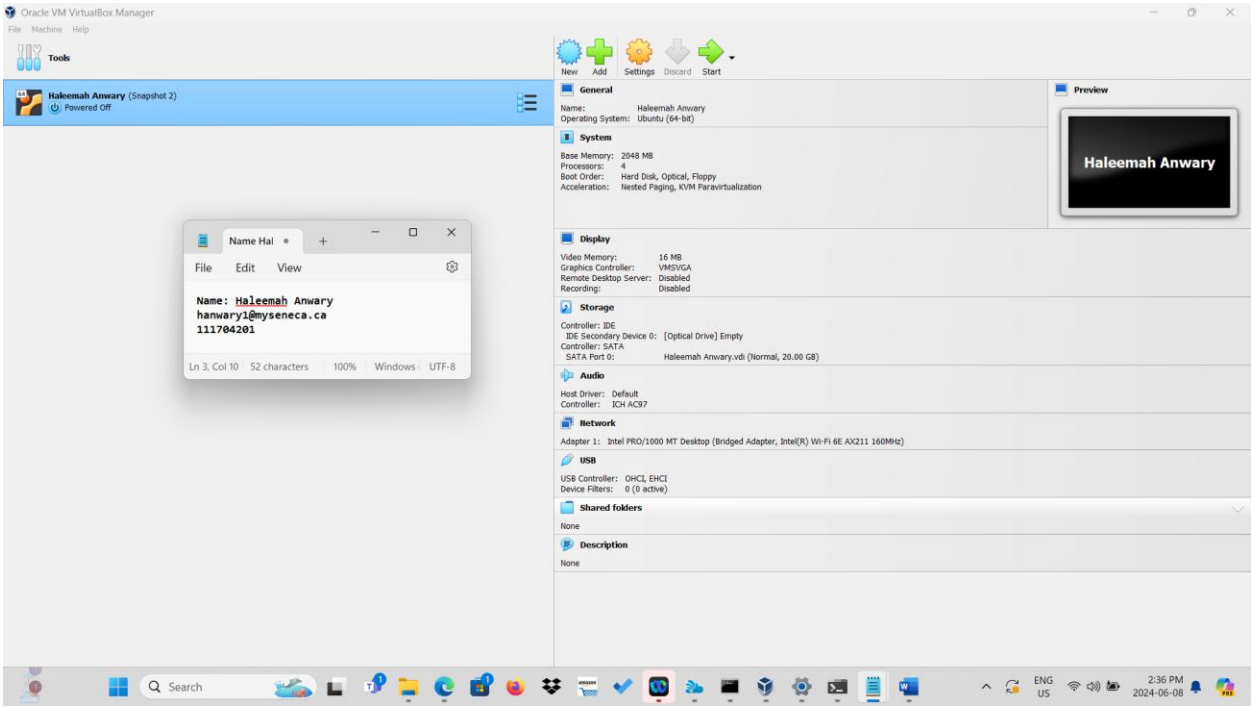
SRT411NAA: Digital Data Analytics

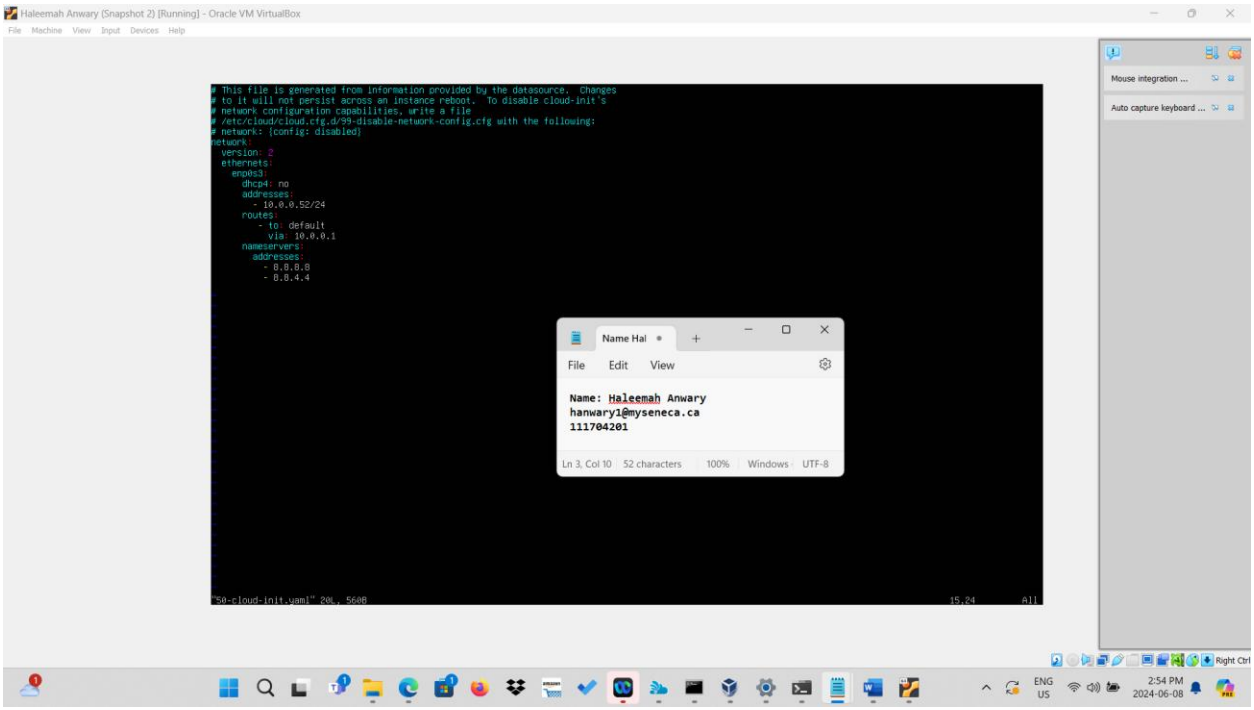
Kosalan Balarajah

Contents

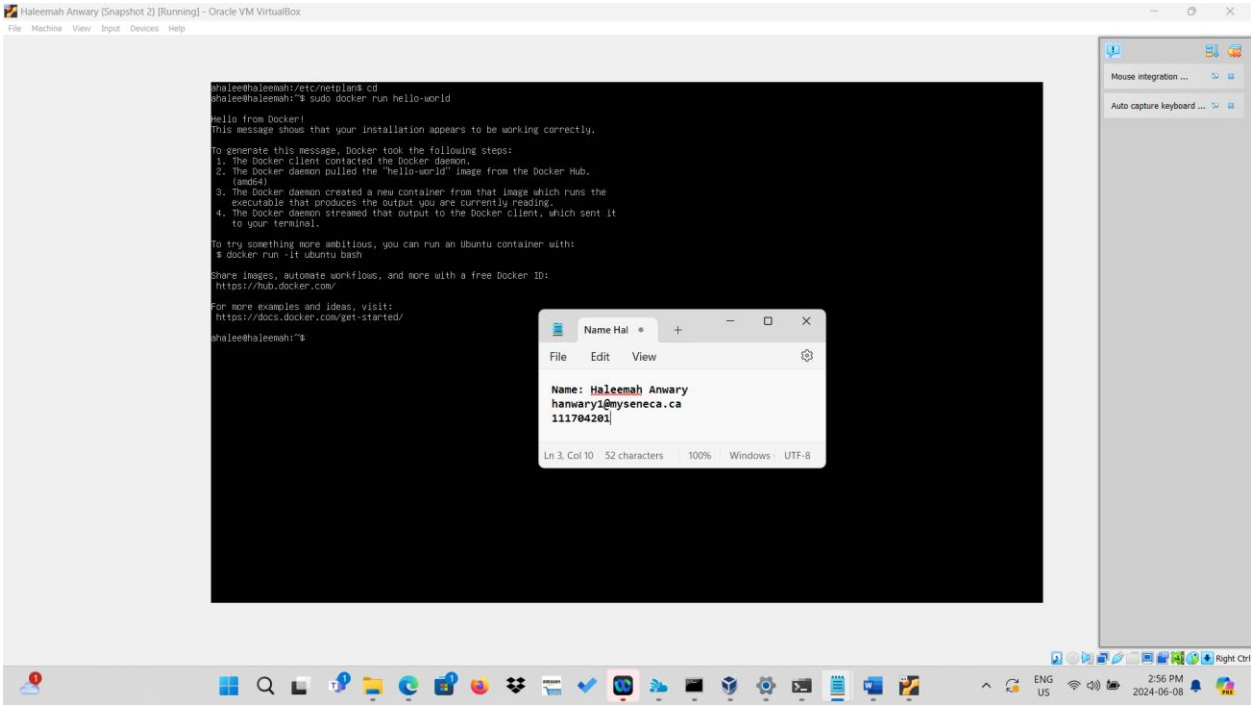
Installing Virtual Box (Task 01 & 2) 3
Installing Docker on Ubuntu Server (Task 03) 4
Install Portainer on Server (Task 04) 4
Access Portainer 5
Connected 5
Run MySQL as Container (Task 05) 6
Accessing MySQL Server (Task 06) 6
Connection Successful 7
Clean up Data (Task 07) 7
Explain and Demonstrate SQL Queries (Task 08) 8
CREATE DATABASE 8
CREATE TABLE 9
INSERT INTO 10
SELECT 11
LEFT JOIN 11
RIGHT JOIN 12
Full JOIN 12
INNER JOIN 13
UPDATE / SET 13
WHERE 14
DELETE FROM 14
GROUP BY 15
ORDER BY 15
NOT IN 16
CREATE INDEX 16
CREATE VIEW 17
CREATE PROCEDURE 18
CALL / PROCEDURE 19
CREATE TRIGGER 19
START TRANSACTION 21
ETL Process Explanation (Task 09) 23

Installing Virtual Box (Task 01 & 2)

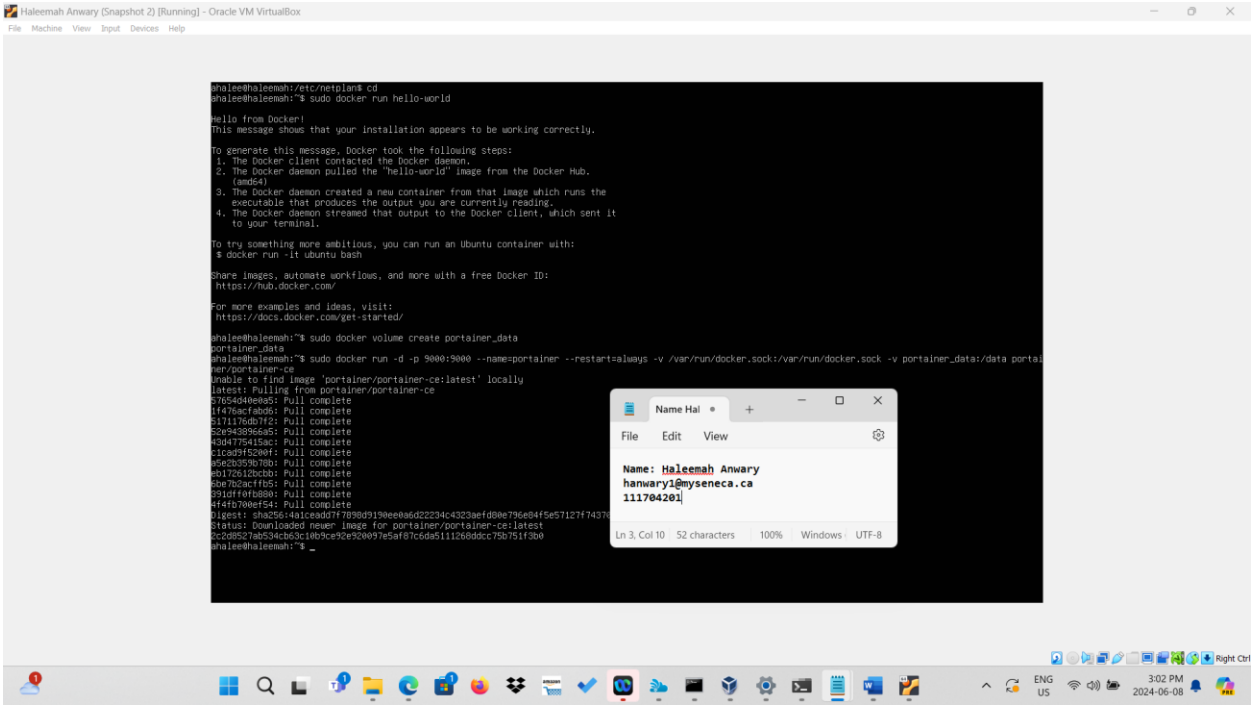




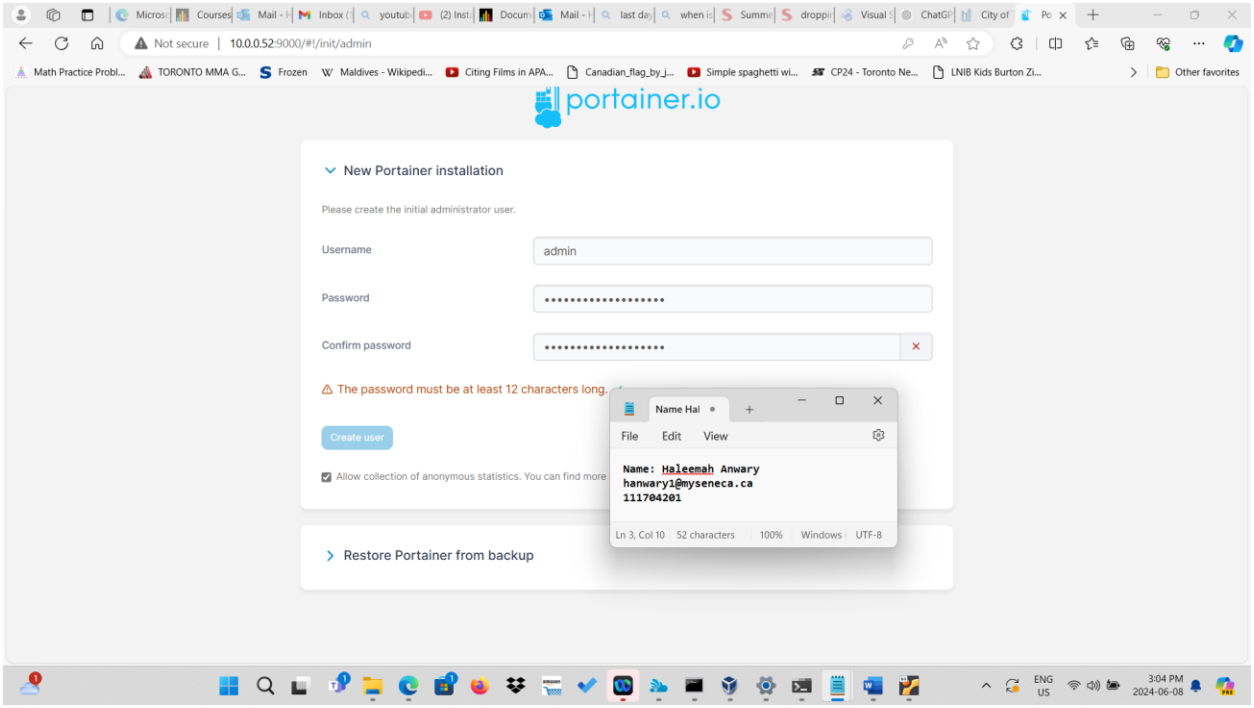
Installing Docker on Ubuntu Server (Task 03)



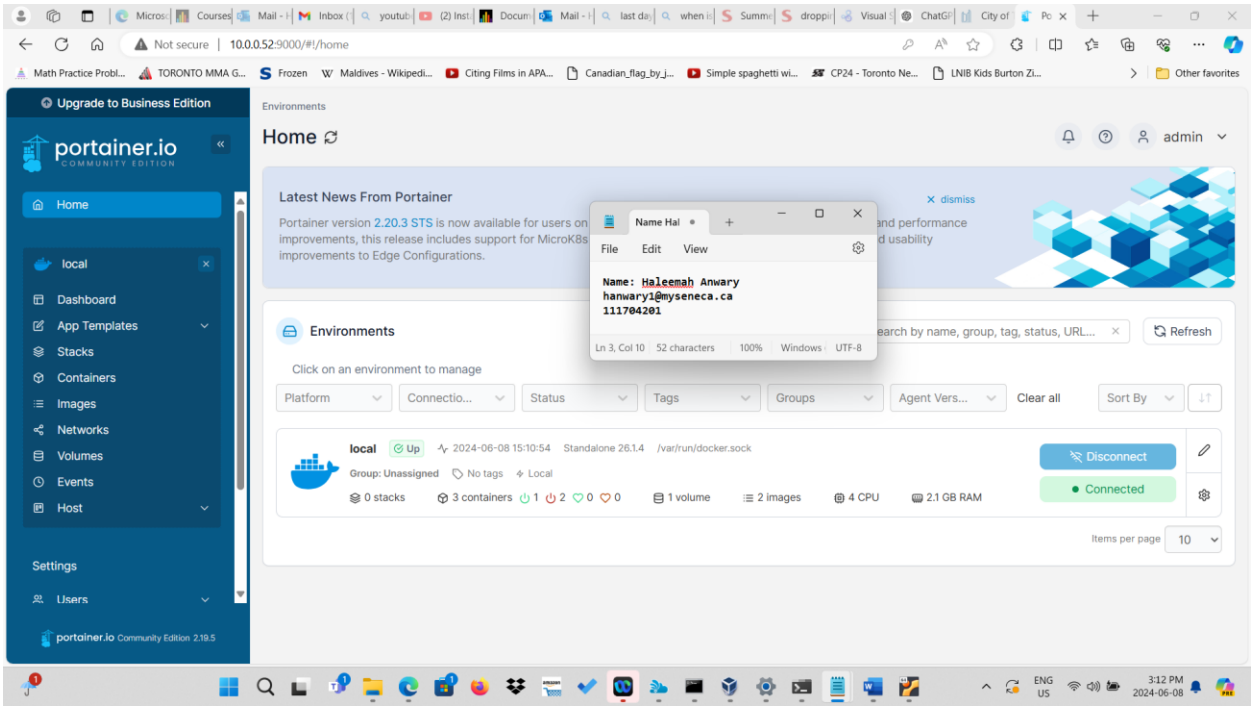
Install Portainer on Server (Task 04)



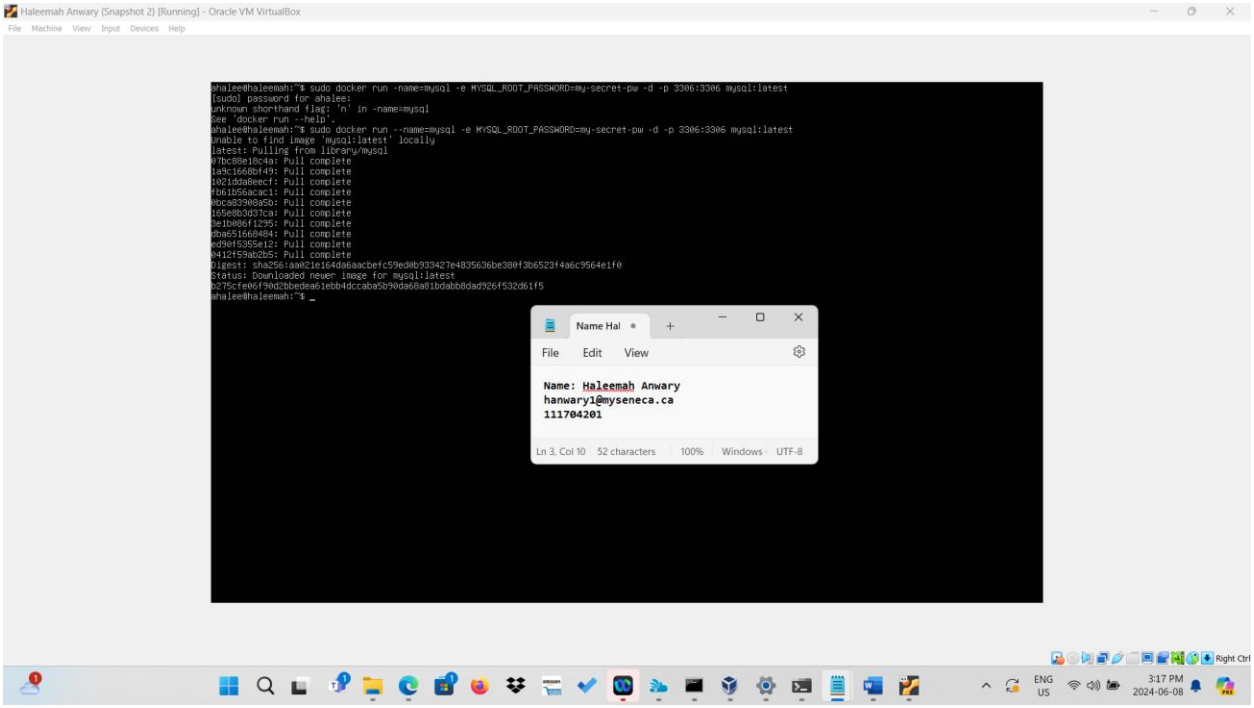
Access Portainer



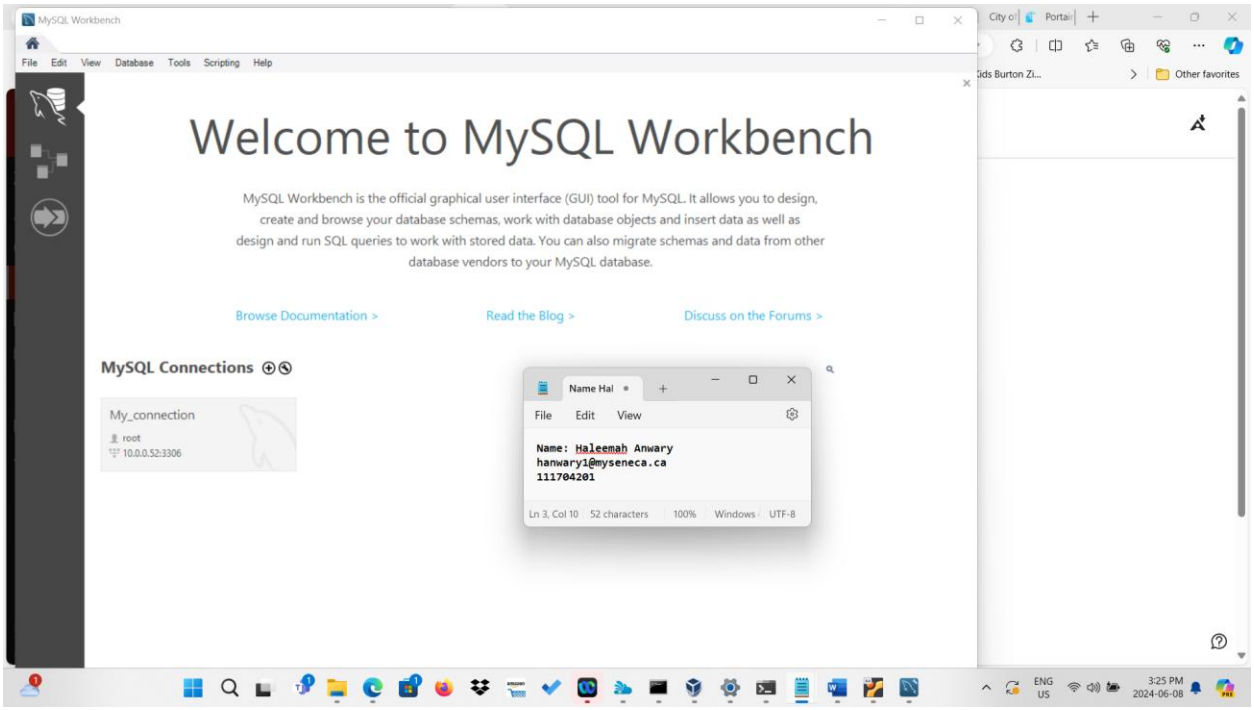
Connected



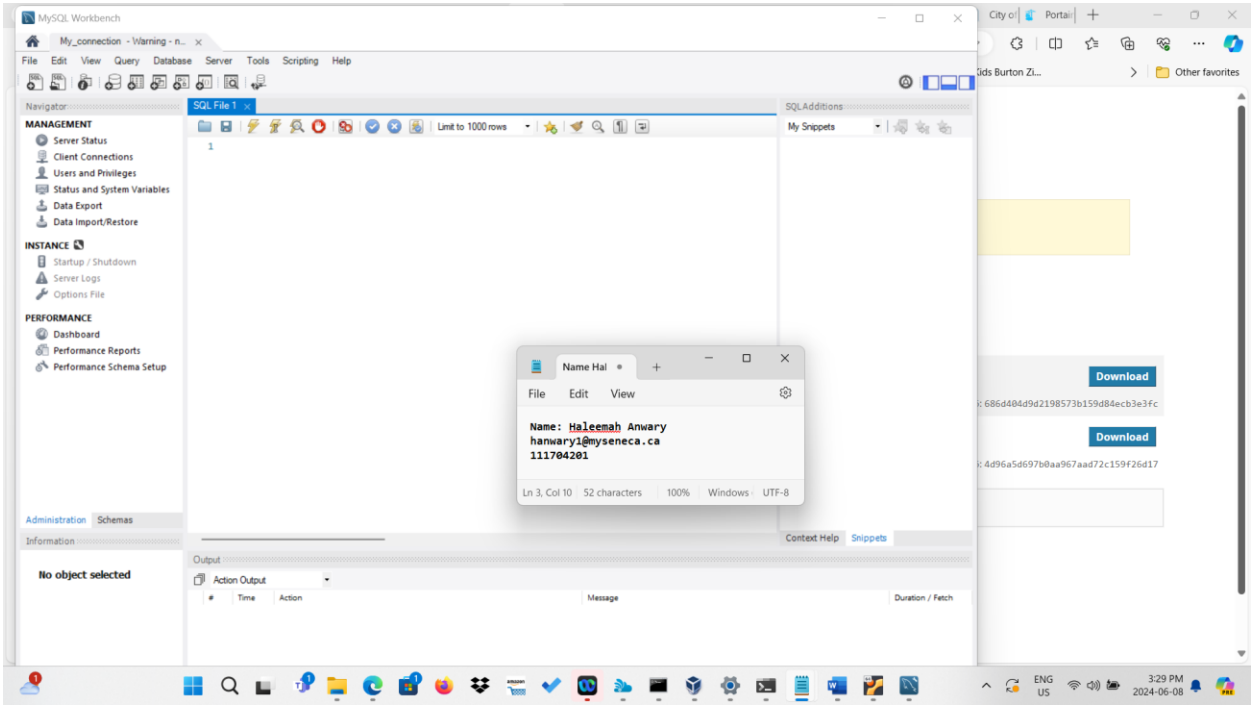
Run MySQL as Container (Task 05)



Accessing MySQL Server (Task 06)



Connection Successful



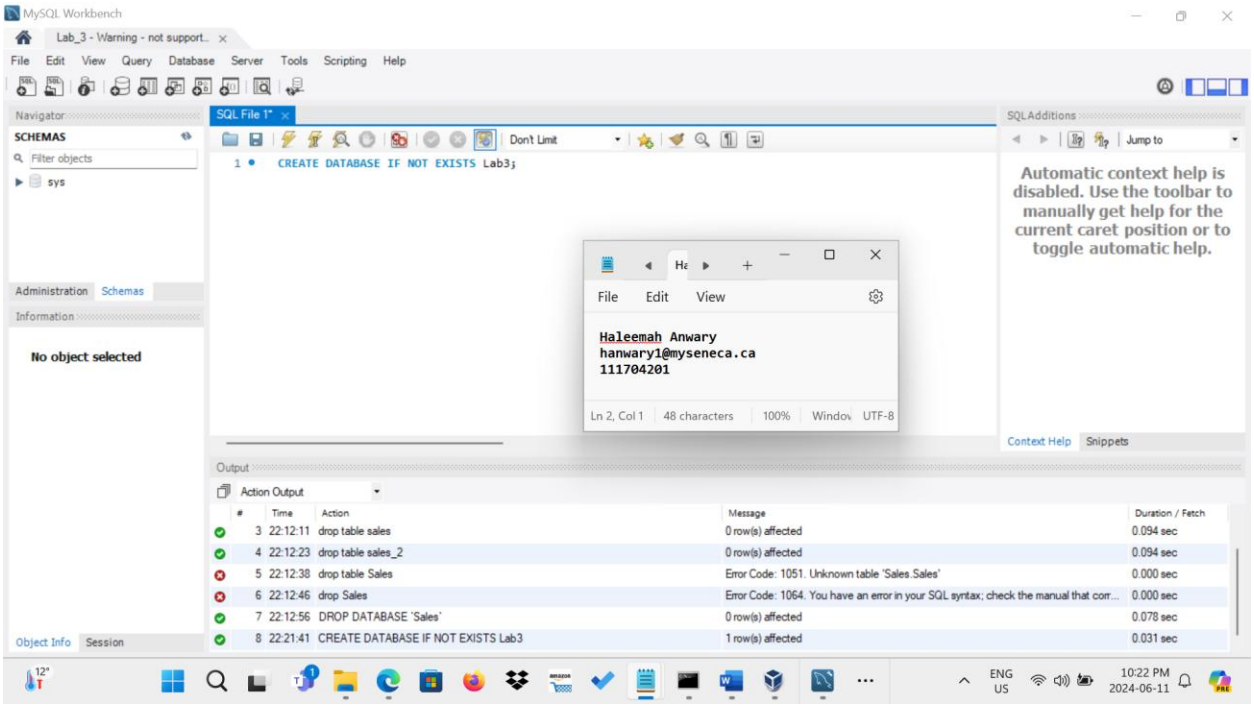
Clean up Data (Task 07)

Cleaning Scope	How its done	Why its Important
Remove duplicate	Under 'Data' there's an option to remove duplicates.	Ensures each record is unique
Standardize Country	=UPPER()	Maintains consistency in queries
Trim spaces Sub_category, State,Product, Age_group	=TRIM()	Removes unwanted characters including extra spaces
Cover Data Types	Format cells	Enhances readability and usability, important for automation and processing, and avoids errors in data entry.
Correct errors (,)	Find&Replace	Remove unwanted characters and spaces. I used it to remove the extra commas.

Explain and Demonstrate SQL Queries (Task 08)

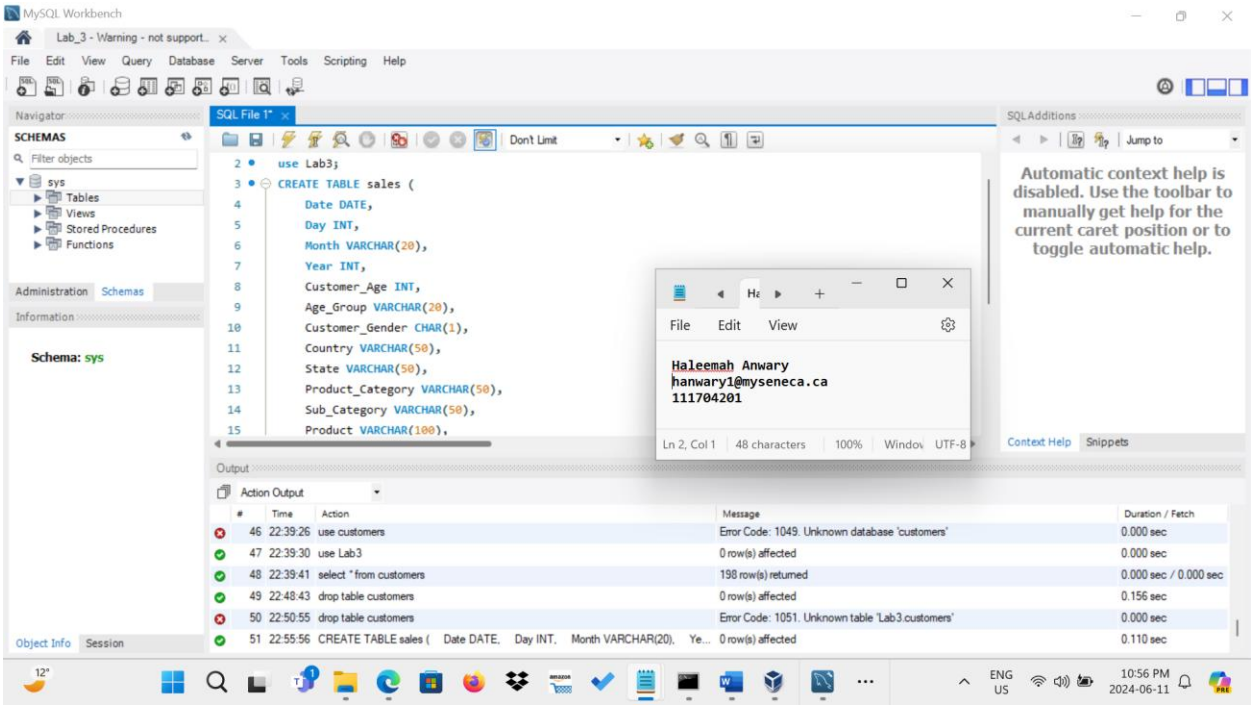
CREATE DATABASE

Creating the Database: The command CREATE DATABASE IF NOT EXISTS Lab3; ensures that the database named Lab3 is created if it doesn't already exist.

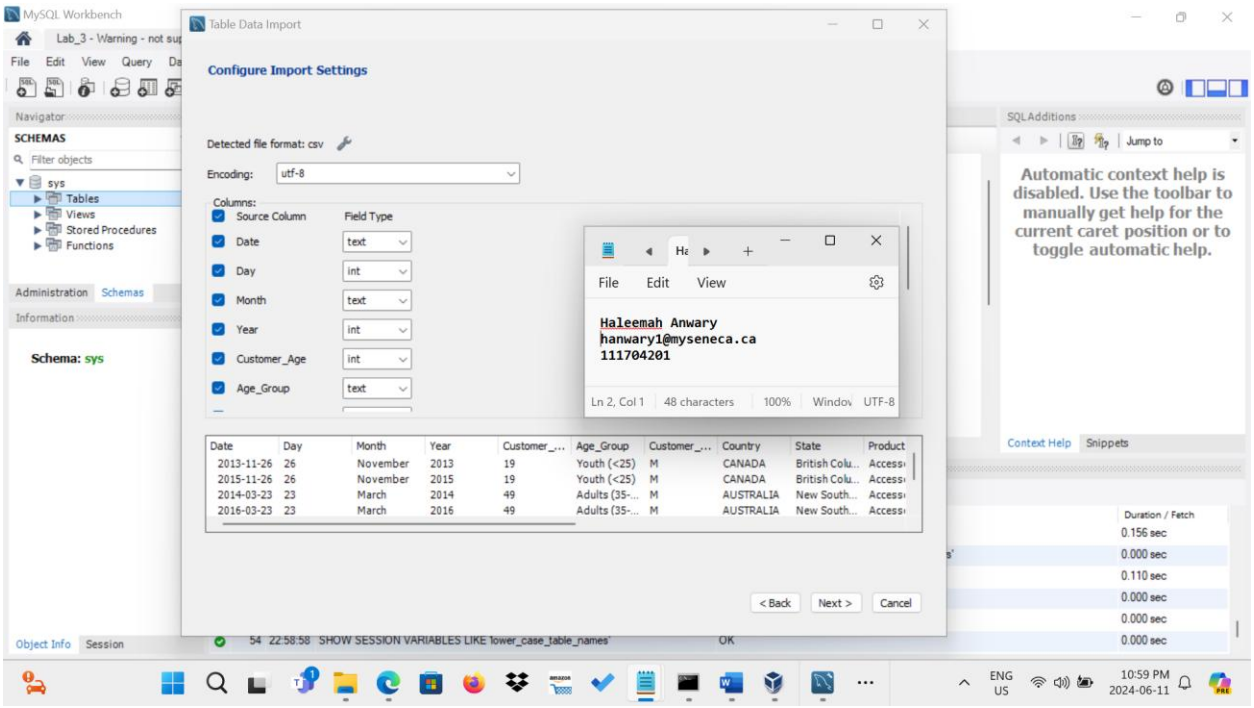


CREATE TABLE

Creating the sales Table: This table is designed to store comprehensive sales data, including details like date, customer demographics, product details, and financial metrics. Each column is appropriately typed for the data it will store (e.g., DATE for Date, VARCHAR for text fields, INT for numeric fields).

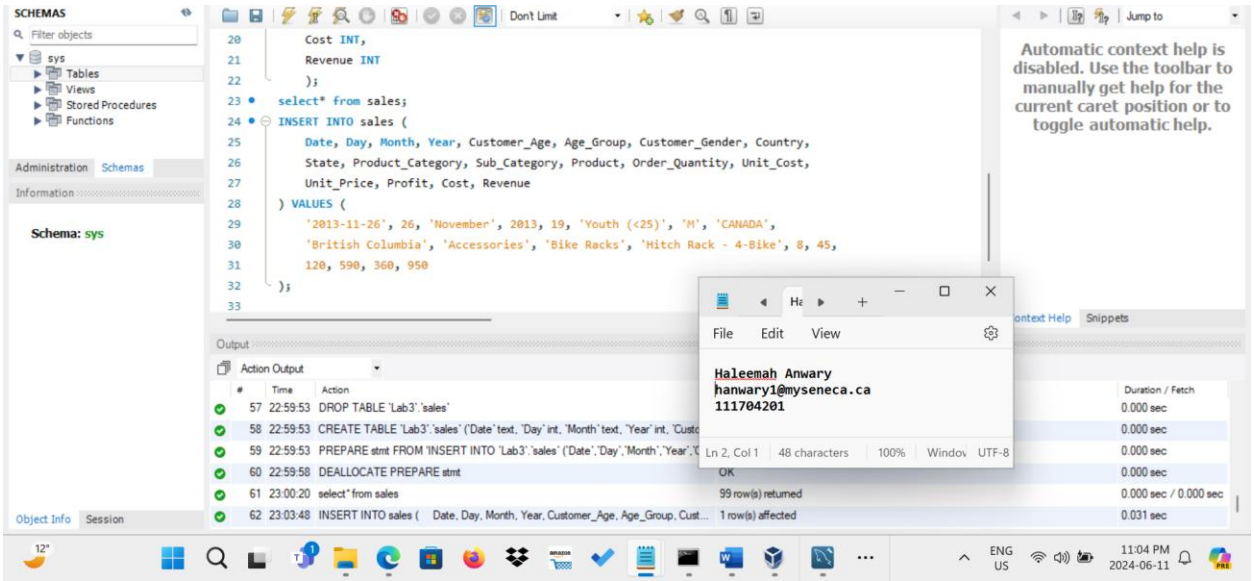


Loading the data into mysql benchmark



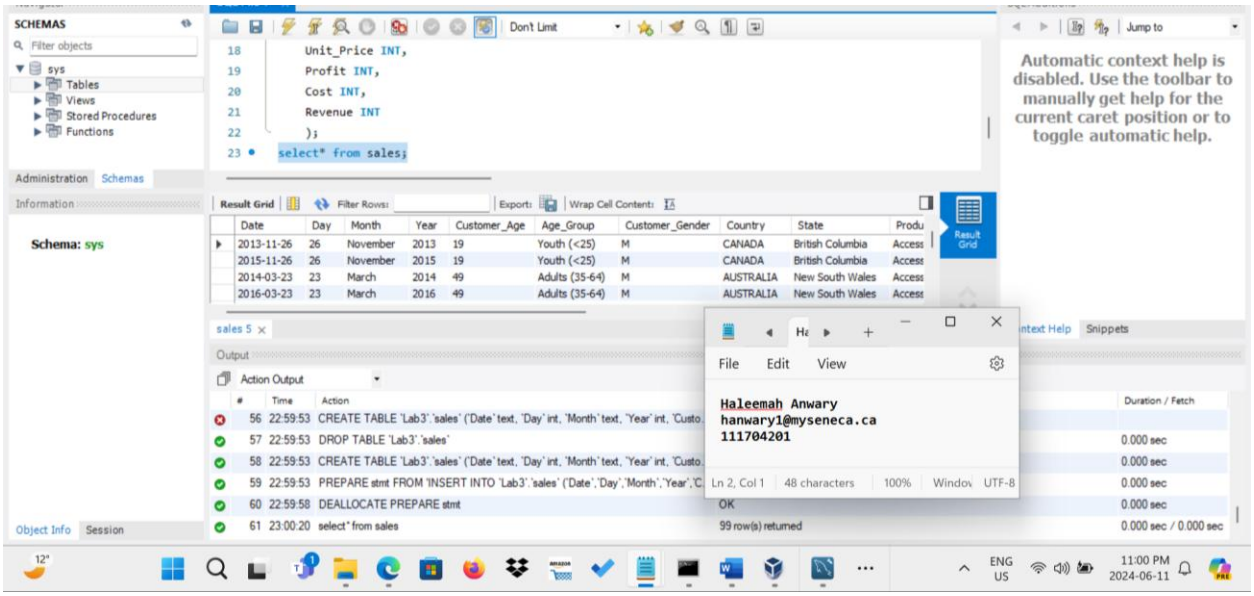
INSERT INTO

Inserting Data into sales: This command inserts a single record into the sales table, demonstrating the format and data types needed for each column.



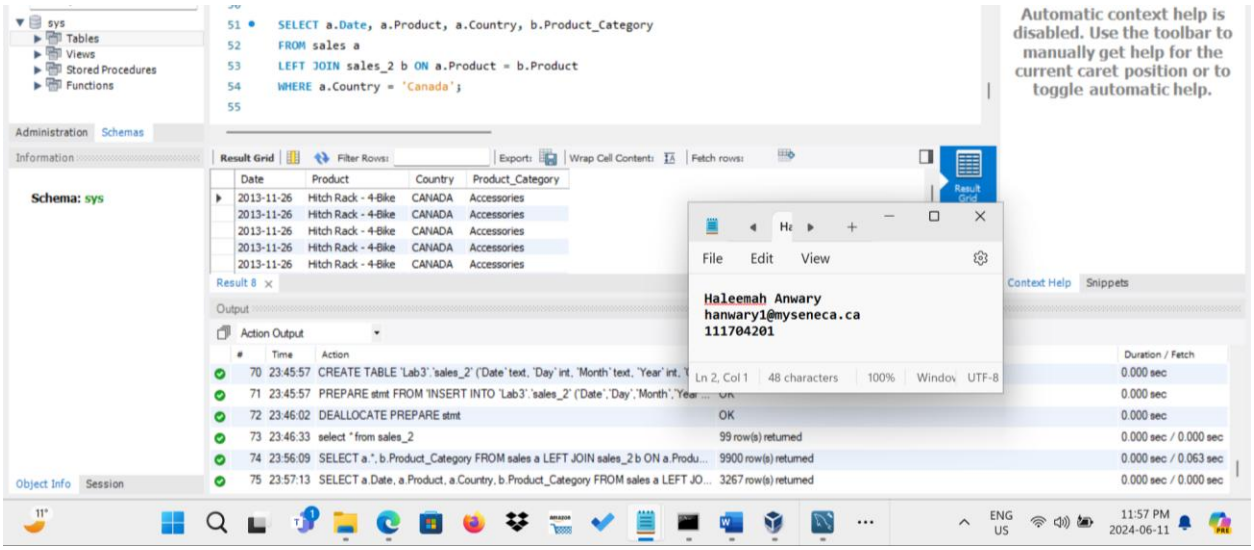
SELECT

Select Query: The SELECT * FROM sales; command is used to retrieve all records from the sales table.



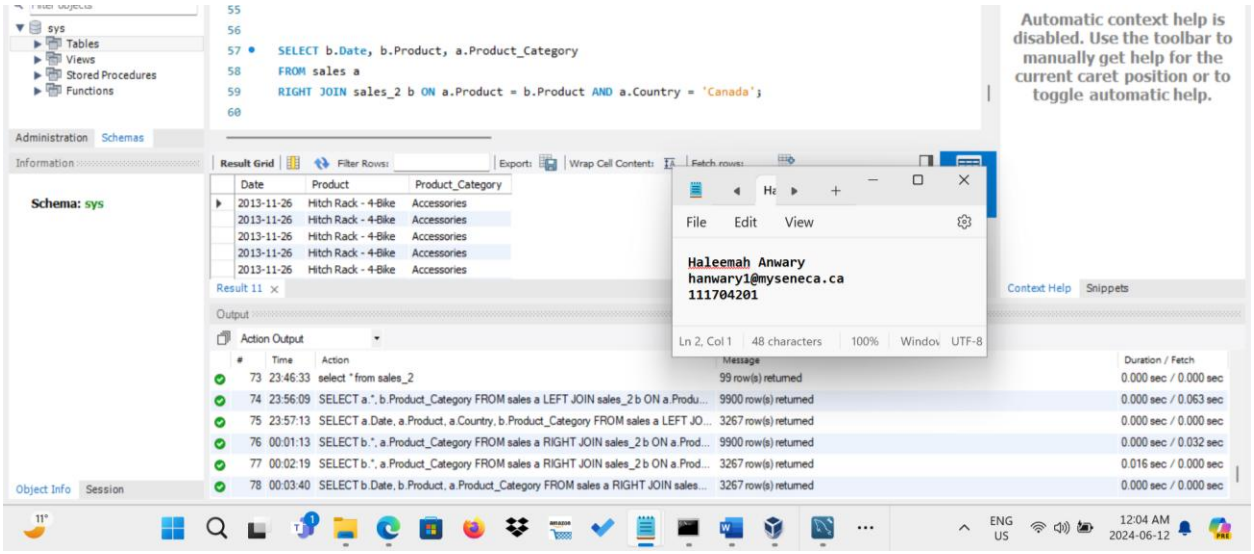
LEFT JOIN

This query performs a LEFT JOIN on the sales and sales_2 tables based on the Product field, but it only includes results where the Country field in the sales table is 'Canada'. The LEFT JOIN ensures that all entries from the sales table (where the country is Canada) appear in the result set. If there's no corresponding entry in the sales_2 table for a particular product, the Product_Category will be displayed as NULL.



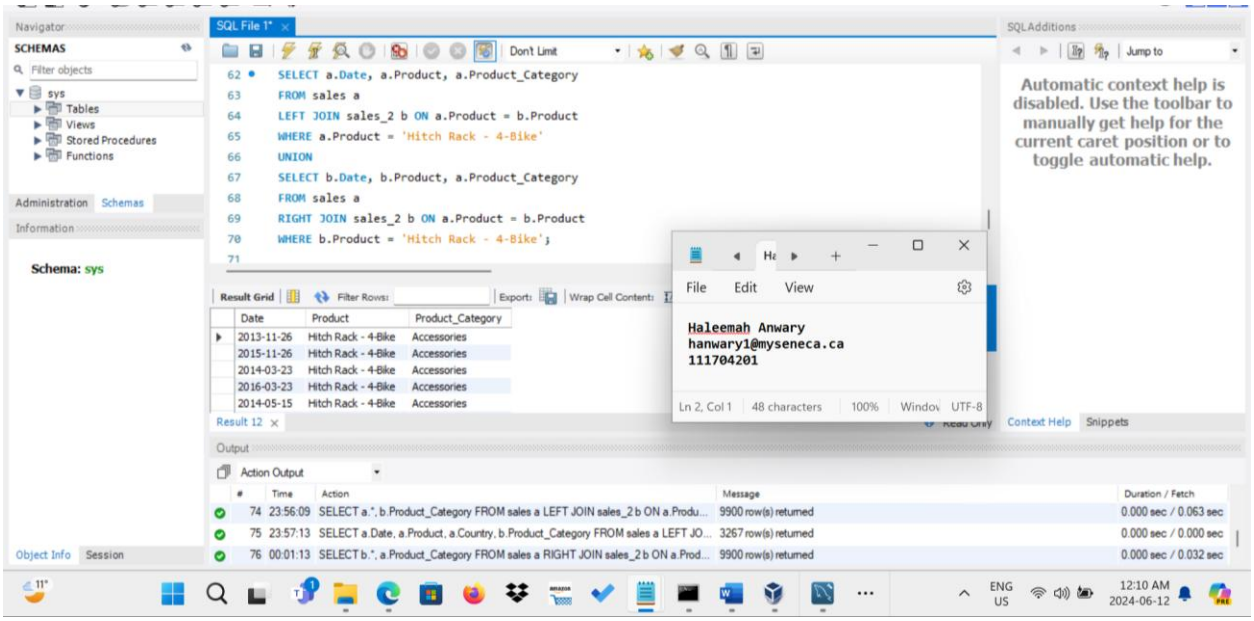
RIGHT JOIN

The result will include all records from sales_2, but for each record, only the Date, Product, and Product_Category fields are displayed. If there's no corresponding Product from 'Canada' in the sales table, the Product_Category will appear as NULL.



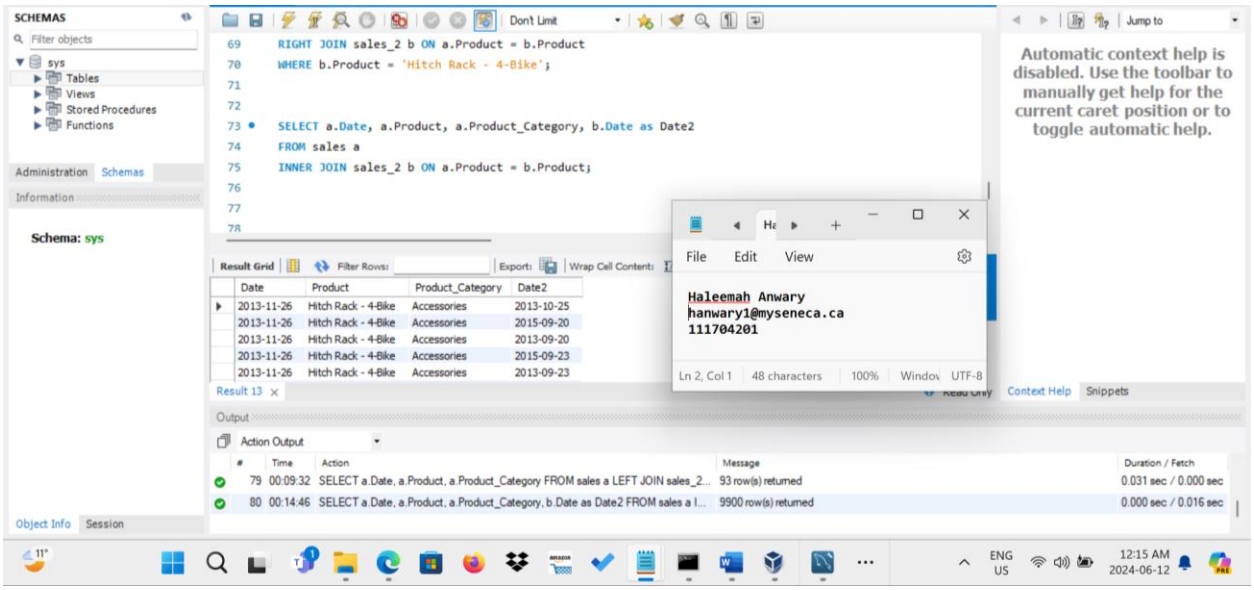
Full JOIN

Full join isn't supported in mysql so we use a combination of left join and right join. Gives matches for both tables, even if there's no matches.



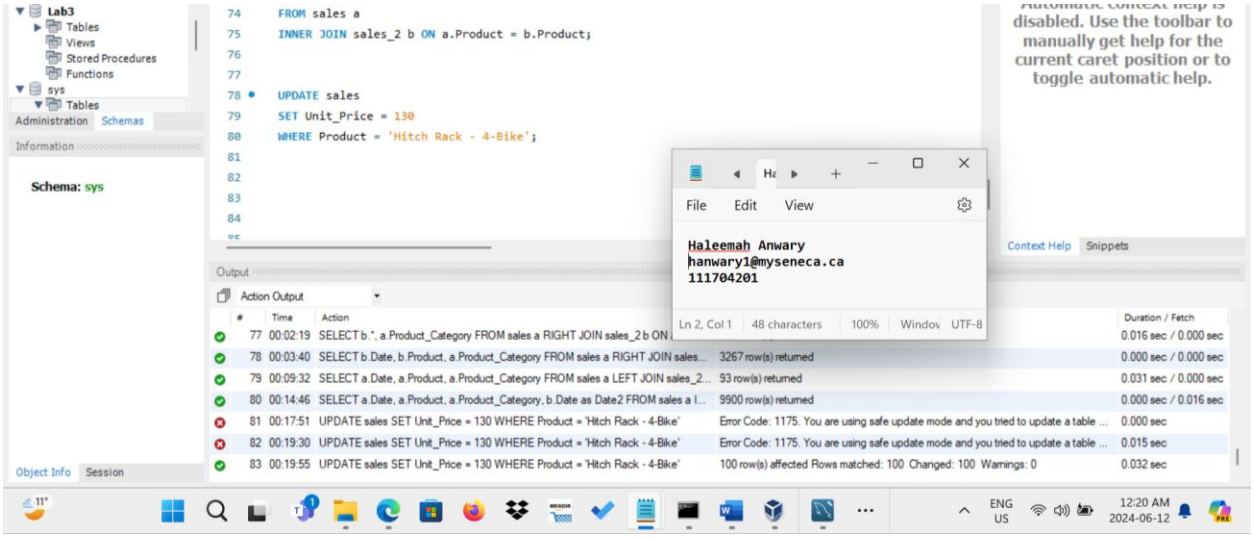
INNER JOIN

This query will produce a list of all the transactions where the same product appears in both tables. If a product listed in one table doesn't have a corresponding entry in the other, it will not appear in the output of this query.



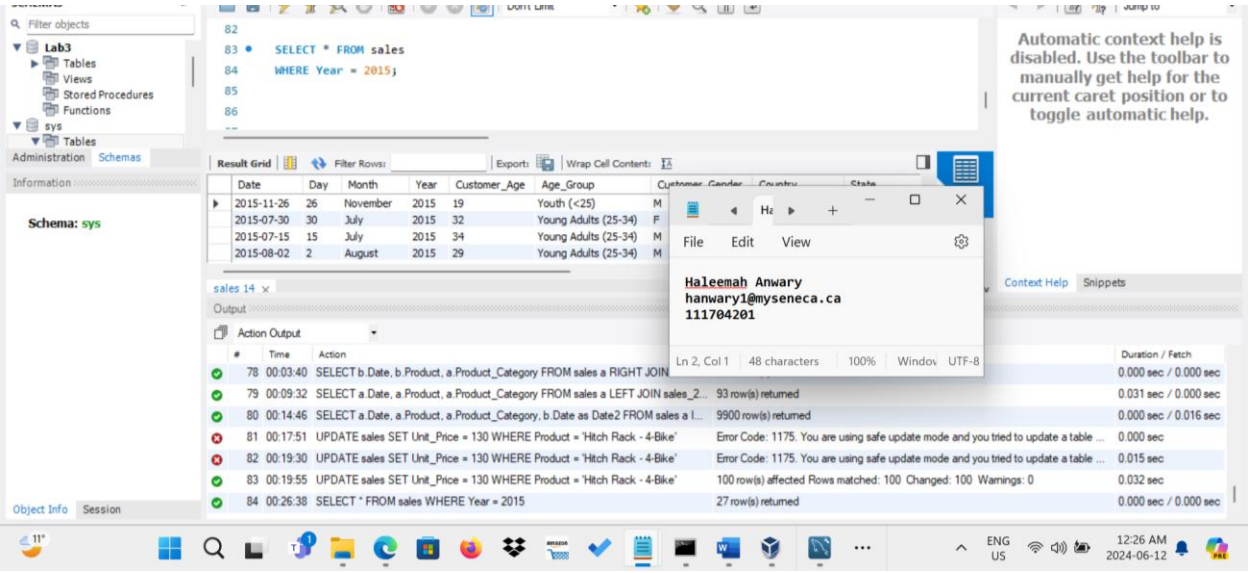
UPDATE / SET

Updates the 'Unit_Price' for 'Hitch Rack - 4-Bike' in the 'sales' table to $130.



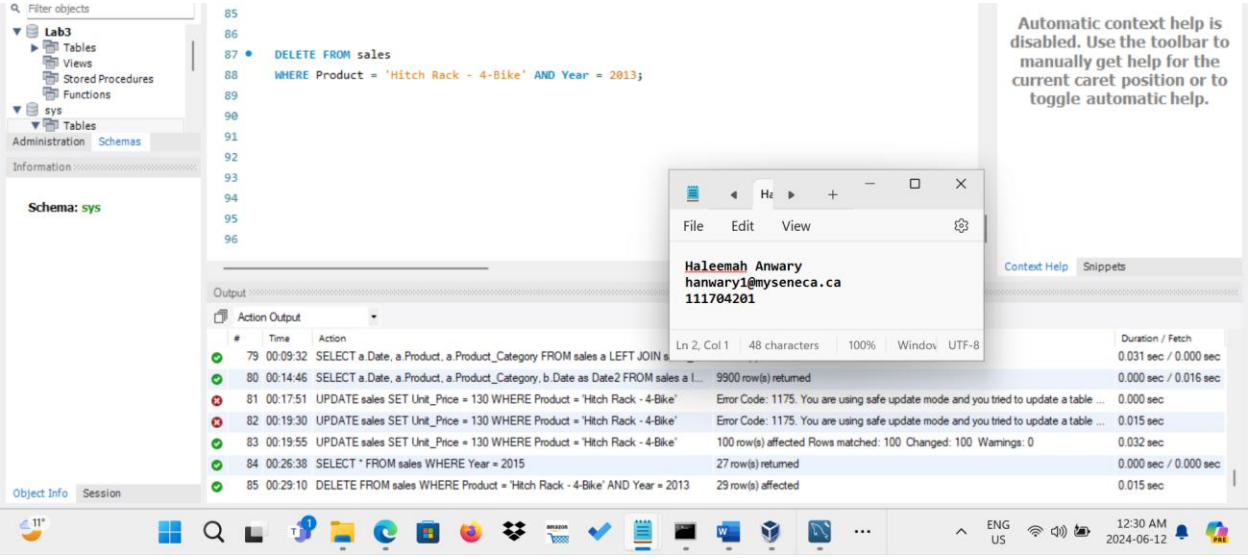
WHERE

A simple WHERE clause in SQL is used to filter records in a database that meet certain criteria. This example retrieves all records from the 'sales' table where the 'Year' is 2015.



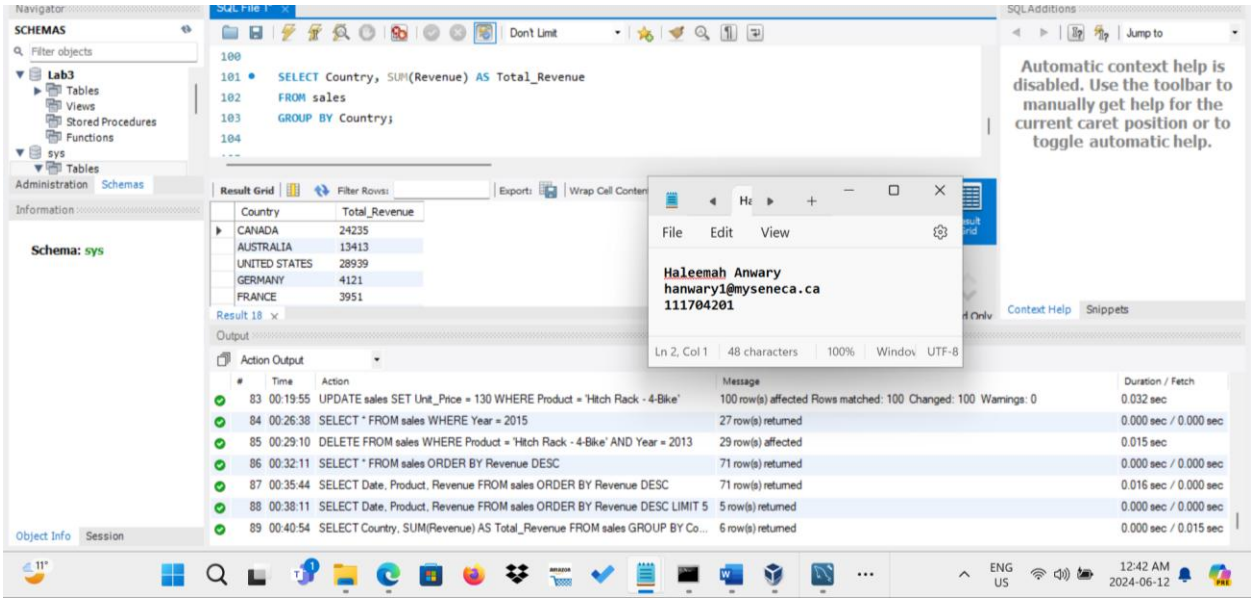
DELETE FROM

A DELETE FROM statement in SQL is used to remove records from a table that meet certain conditions. This example deletes records from the 'sales' table where the 'Product' is 'Hitch Rack - 4-Bike' and the 'Year' is 2013.



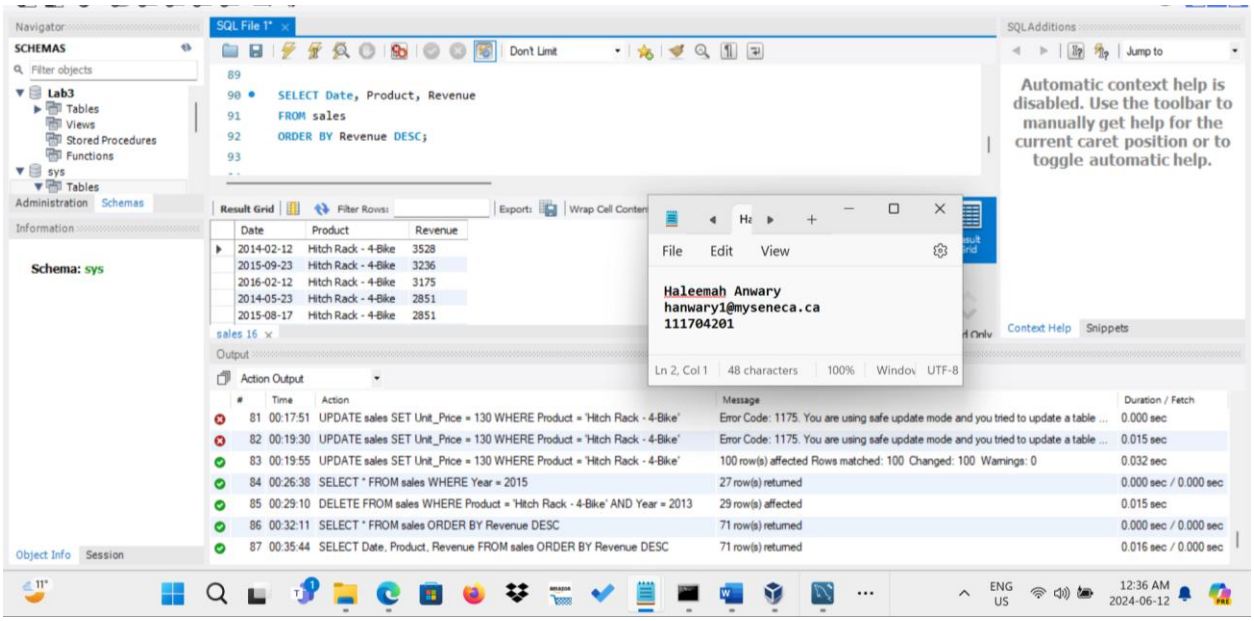
GROUP BY

A GROUP BY statement in SQL is used to aggregate data based on one or more columns. In this example it calculates the total revenue for each country in the 'sales' table.



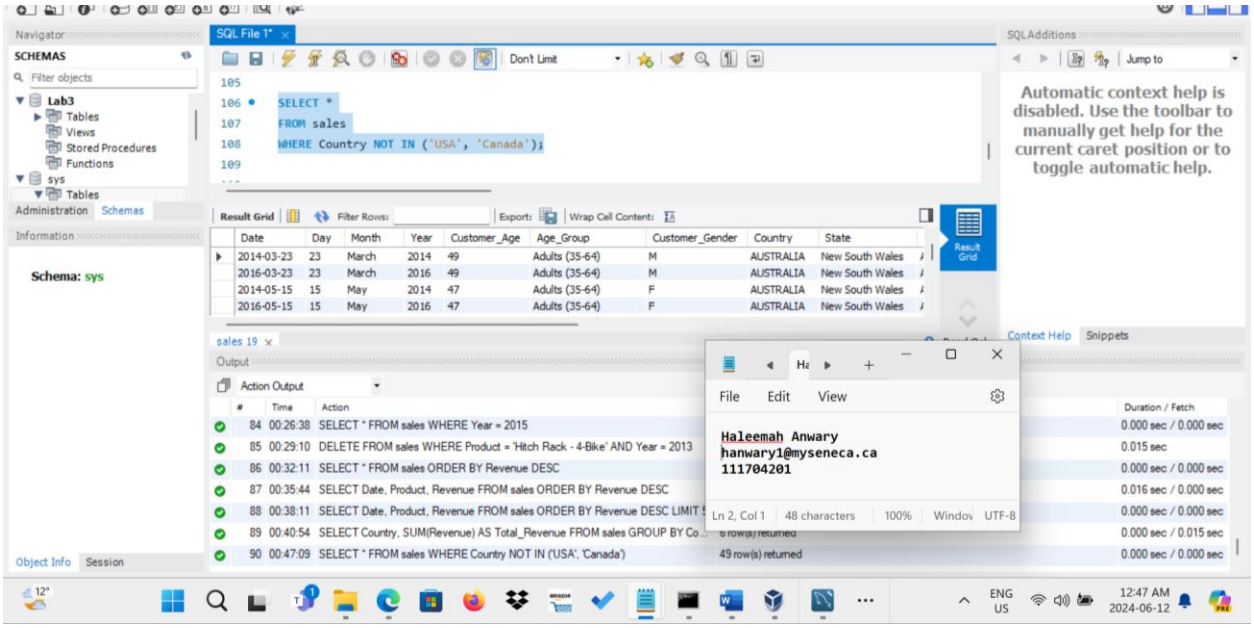
ORDER BY

An ORDER BY statement in SQL is used to sort the results returned by a query in either ascending or descending order based on one or more columns. This example Retrieves the 'Date', 'Product', and 'Revenue' from the 'sales' table and orders them by 'Revenue' in descending order



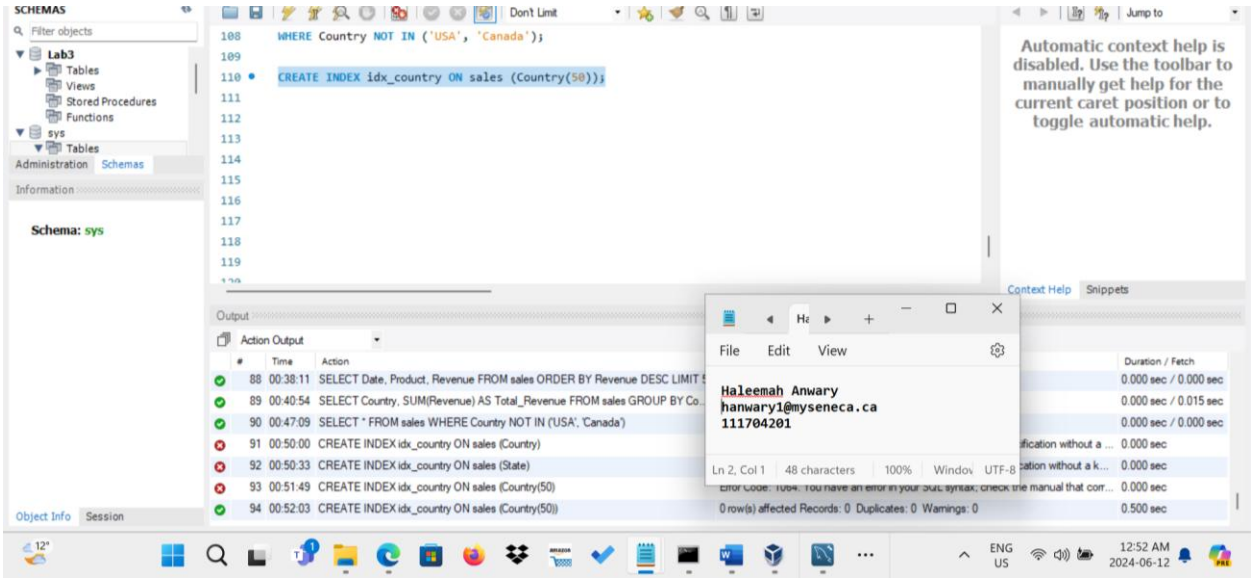
NOT IN

A NOT IN query in SQL is used to filter out records that match a list of specified values. This example retrieves all records from the 'sales' table where the 'Country' is not 'USA' or 'Canada'.



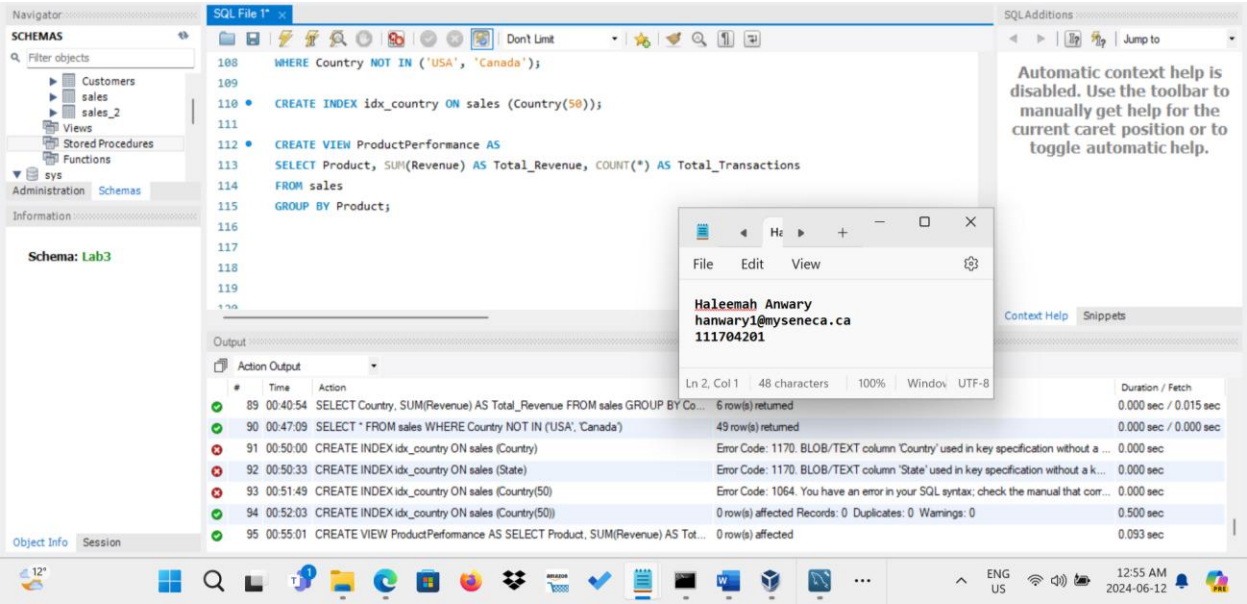
CREATE INDEX

Creating an index in SQL can significantly improve the performance of queries by reducing the amount of data scanned during searches and speeding up data retrieval. This example creates an index named 'idx_country' on the 'Country' column in the 'sales' table.

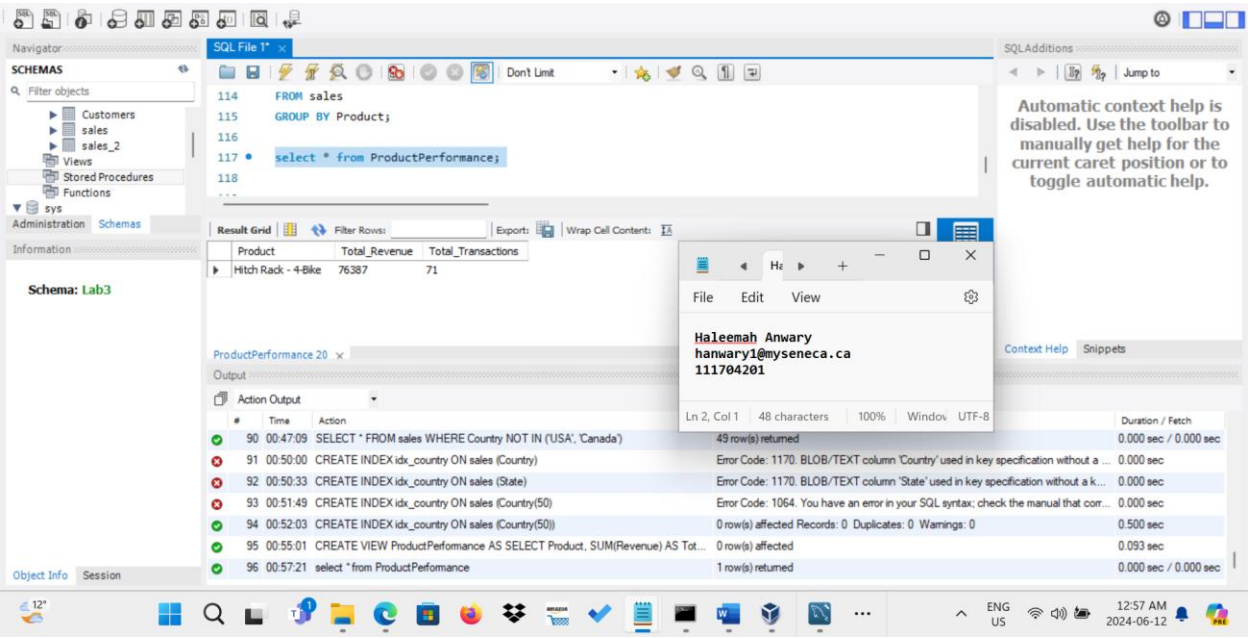


CREATE VIEW

Creating a view in SQL can help simplify complex queries and provide a readable subset of data for repetitive analysis. This example creates a view named 'ProductPerformance' that shows total revenue and number of sales transactions for each product.

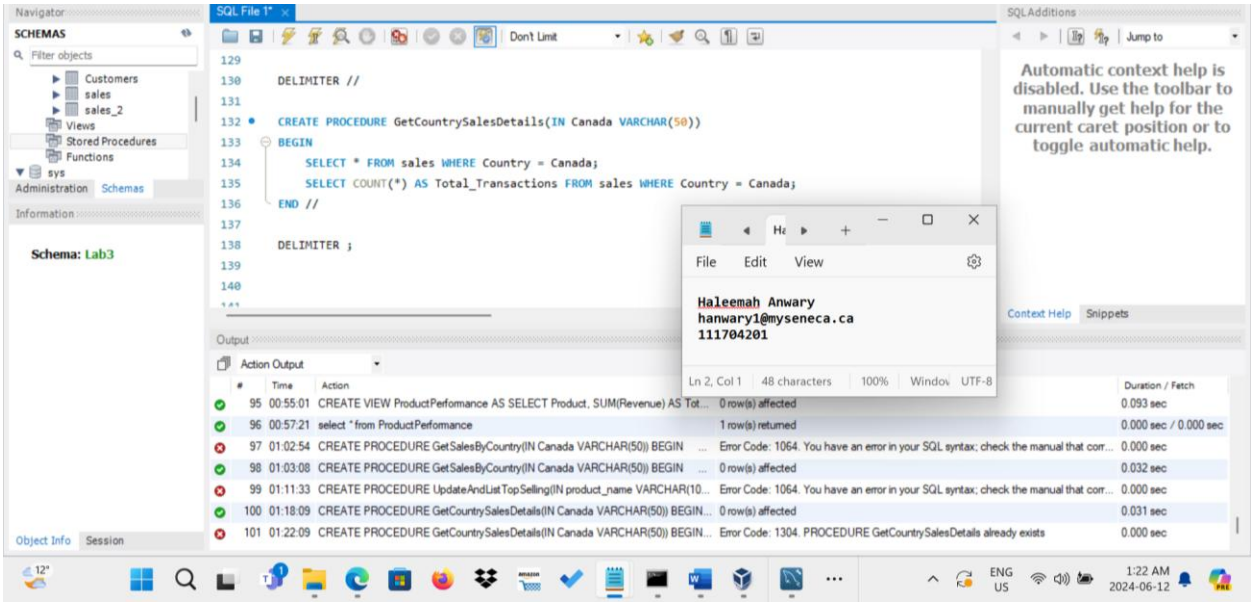


I can use this view anytime.



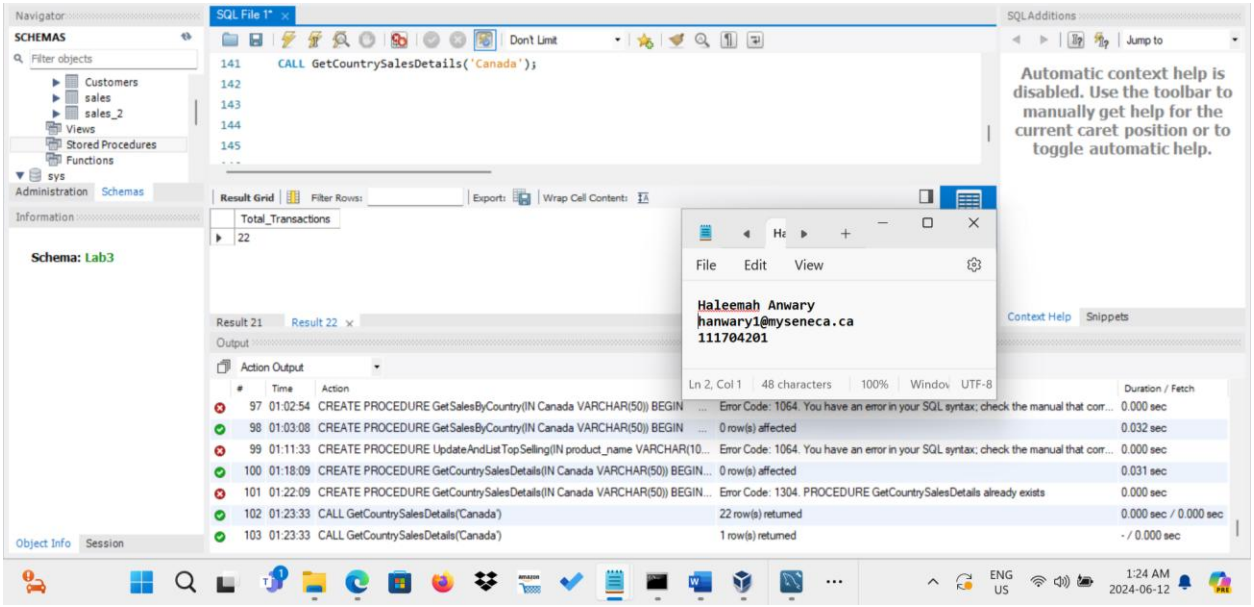
CREATE PROCEDURE

Creating a stored procedure in SQL provides a way to encapsulate SQL queries for reuse and can be particularly useful for operations that are performed frequently. This example creates a stored procedure named 'GetCountrySalesDetails' that retrieves sales records and counts transactions for a given country. It retrieves all sales records for the specified country. Second query inside Counts the total number of sales transactions for the specified country. DELIMITER // sets a new delimiter for the session, allowing the use of semicolons (;) within the stored procedure without signaling the end of the command. After defining the procedure, DELIMITER ; resets the delimiter back to the semicolon, returning it to the default state for normal SQL operations.



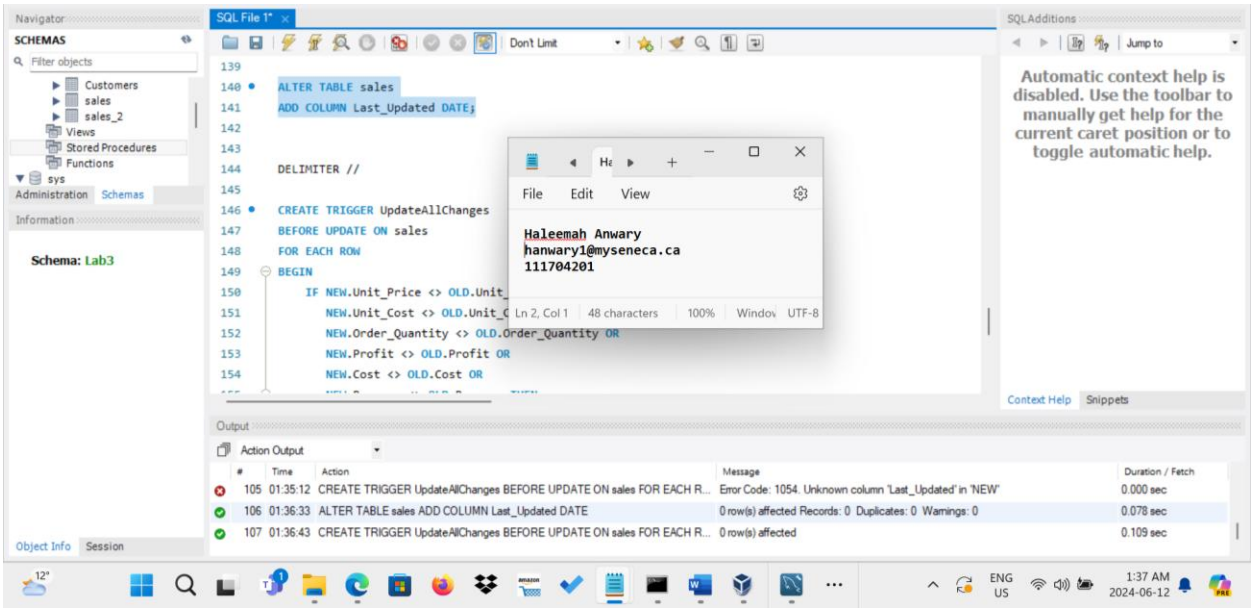
CALL / PROCEDURE

Calls the 'GetCountrySalesDetails' procedure for 'Canada' to retrieve sales data and count transactions.

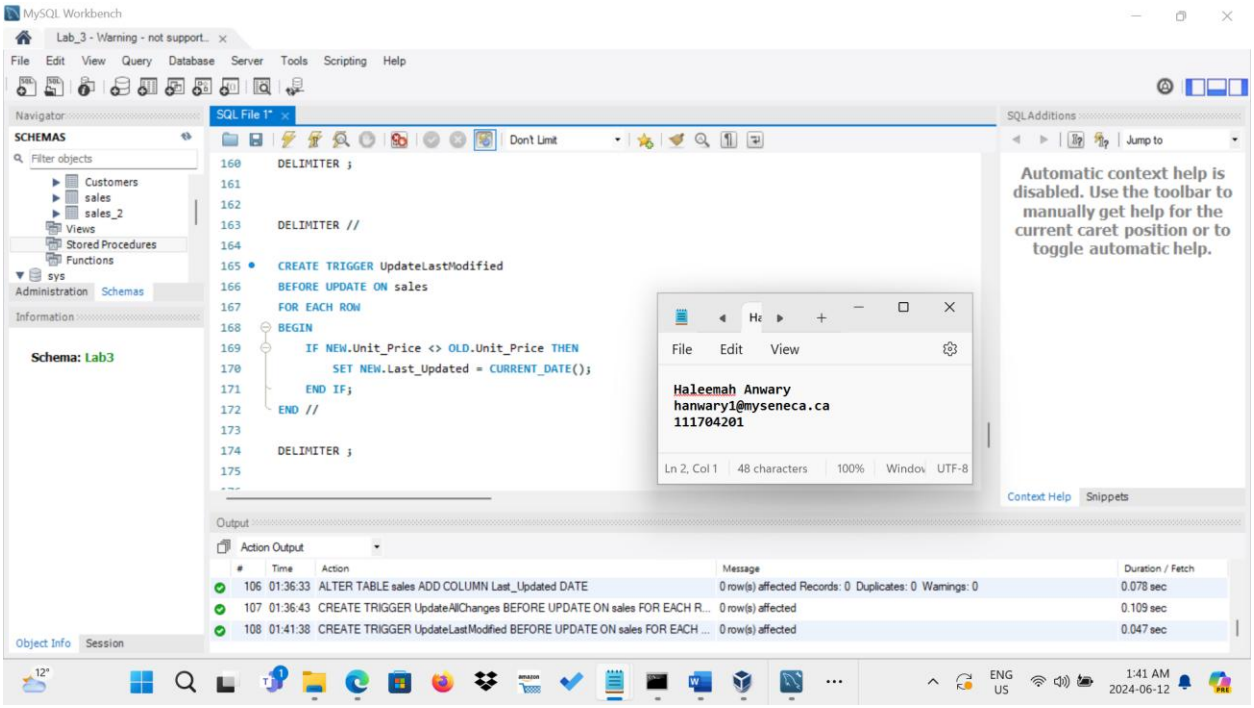


CREATE TRIGGER

I first added another column names ‘Last_updated’

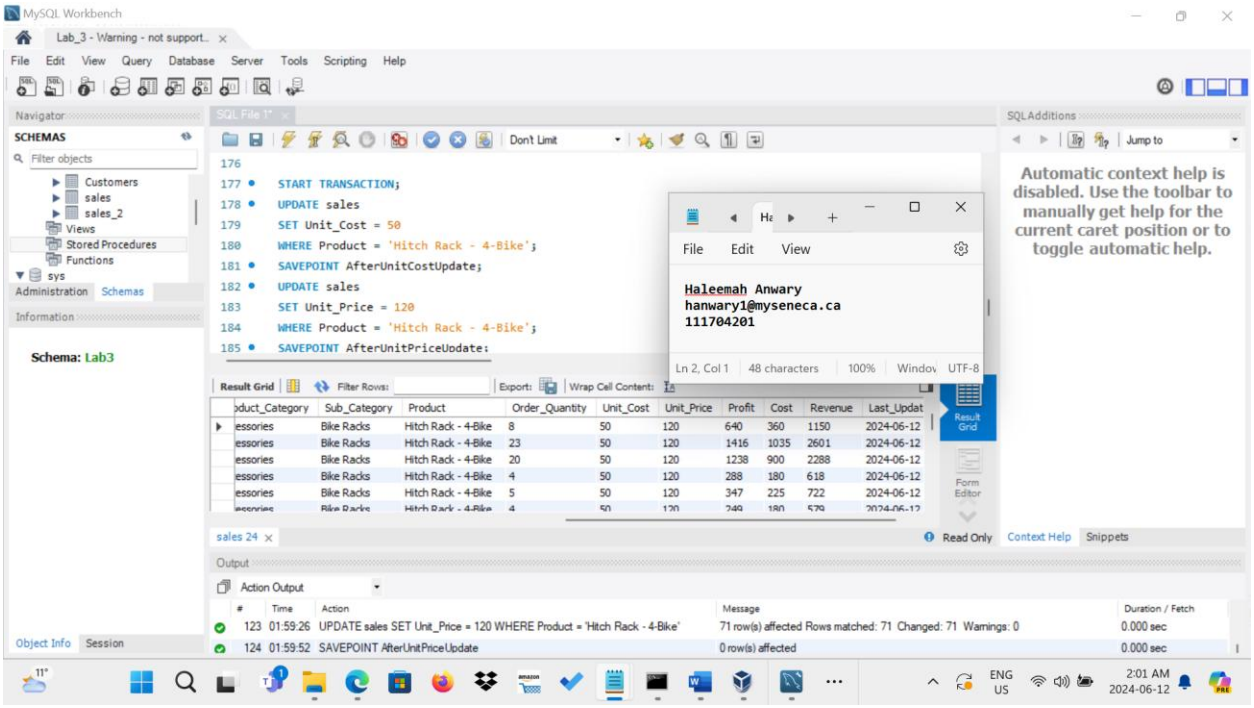


Creating a trigger in SQL allows you to automatically perform actions in response to specific changes in your database. The trigger is set to activate BEFORE UPDATE on the sales table, meaning it runs before any update operation is finalized on this table. FOR EACH ROW indicates that the trigger will examine each row that is about to be updated. SET NEW.Last_Updated = CURRENT_DATE(); updates the Last_Updated field to the current date if the condition is met.

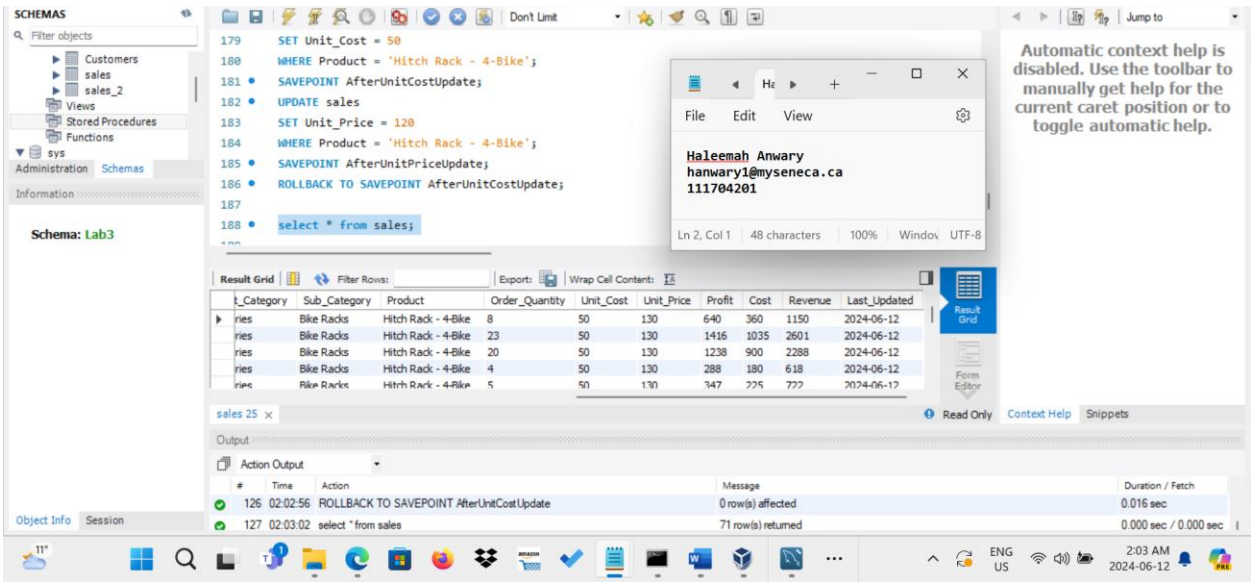


START TRANSACTION

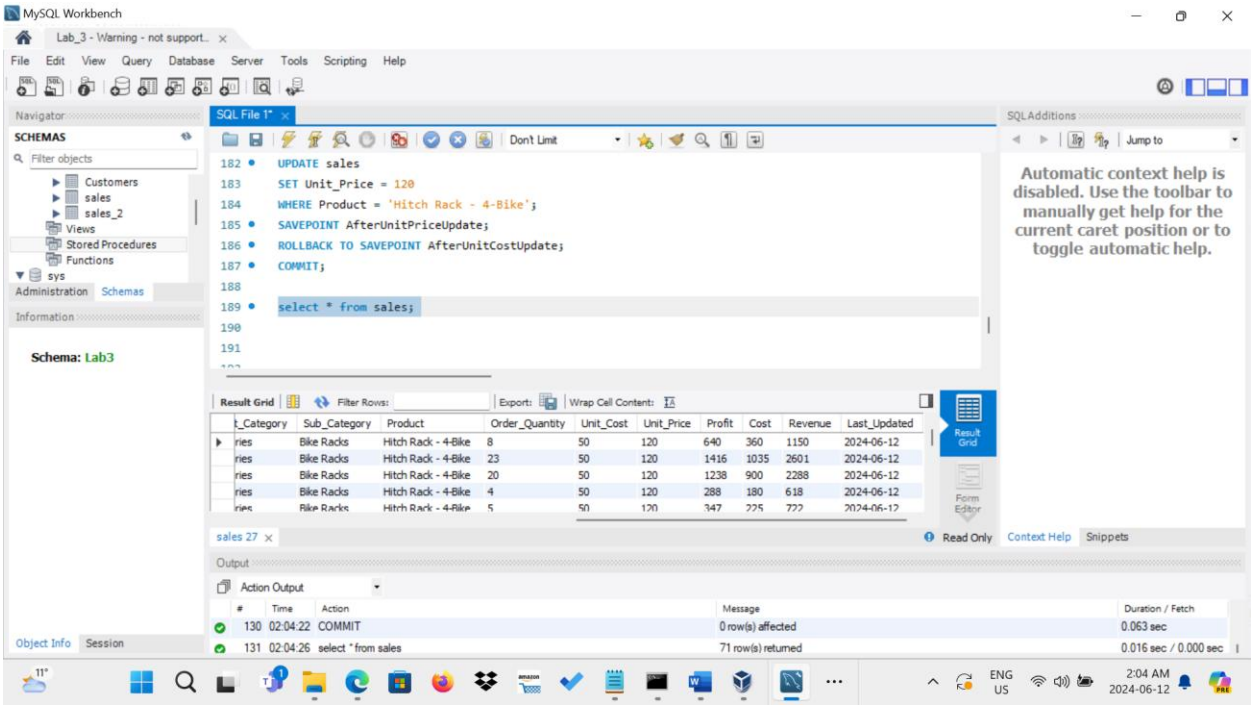
Starts a transaction for updating sales information and update the unit cost for a specific product. Here I've created a save point where I can rollback if I make a mistake later. After running these two updates, I see that the unit_cost has changed to 50 and unit_price to 120.



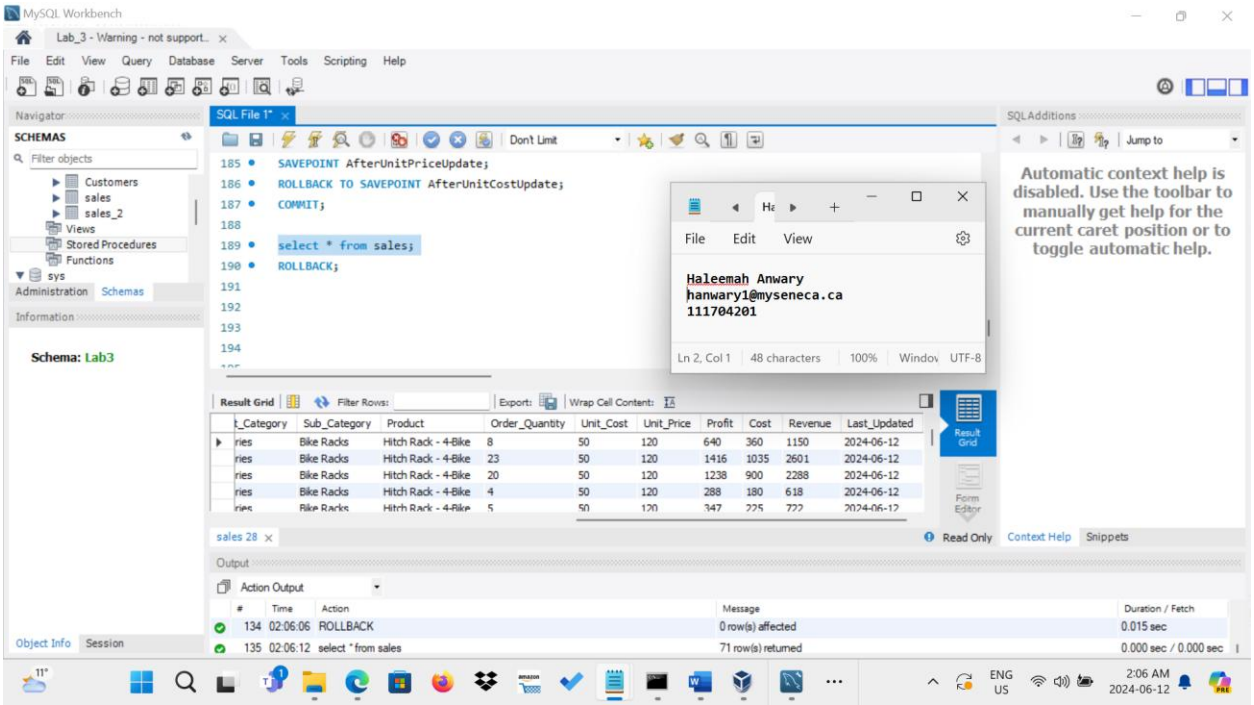
I rolled back to savepoint "AfterUnitCostUpdate" and ran the select statement again. We see that unit_price Is back to 130.



Here, I ran both updates again and used commit, this makes the changes permanent. We can see that we are back to updated unit_price and unit_cost.



After we commit, we cannot rollback that's why changes are still present after running rollback.



ETL Process Explanation (Task 09)

Extract: This phase involves efficiently retrieving data from various sources while maintaining data integrity and source system performance.

Transform: Data is cleansed, normalized, and formatted during this phase to ensure reliability and relevance for subsequent analysis.

Load: Transformed data is transferred into a database optimized for quick access and analysis.

Importance of ETL in Data-Driven Security

ETL consolidates diverse data into a unified format, crucial for streamlined analysis and rapid threat detection. Transforming data ensures its quality and integrity, vital for effective security measures and regulatory compliance.

Learning Outcomes and Professional Relevance

Mastering SQL and the ETL process equips me with essential skills for designing and managing secure, efficient databases, developing complex SQL queries for automated threat detection, and optimizing data handling for real-time analytics. These capabilities are crucial for quick incident responses and maintaining compliance with security standards, making SQL and ETL foundational skills in cybersecurity and enhancing my ability to protect data assets and make informed security decisions.